

Distributed Machine Learning Inference

Omar Attaalla
Hao Su

March 2024

1 Introduction

Machine Learning has been a popular topic of discussion in the computing space over the past few years. With many new architectures being designed and implemented and many use cases for machine learning (ML) being discovered, the speed at which ML occurs is incredibly important. Many ML models are presented to frontend clients through the server/client architecture, where the server makes inference using a model and passes the results back to the client. With the potential for a server to need to handle millions of clients, there needs to be a scalable architecture that significantly improves the speed at which inference is performed.

In this paper, we discuss a scalable architecture that makes use of distributed computing to perform inference on an arbitrary model. This architecture is able to handle multiple clients through the use of multiple servers. In addition, we will consider the performance differences between our architecture and a single server setup in many different situations, including a single client scenario and a multiple client scenario. In particular, we discuss an image based inference model which requires the clients to send images to the server for inference to be performed.

2 Approach

2.1 Distributed Architecture

The distributed architecture that we designed is shown below in Figure 1. Each client passes images to a load balancer that reroutes the requests to different Master nodes. These Master nodes perform distributed inference by utilizing its worker nodes. We will further discuss how the clients, load balancer, and master nodes are designed to maximize performance. We will also discuss possible points of failure in the architecture and possible areas that can be improved to garner improved performance.

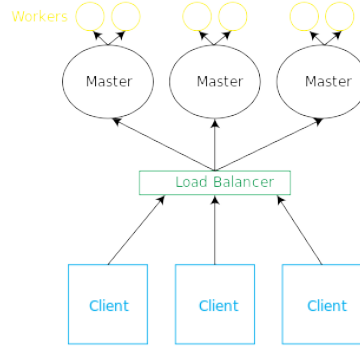


Figure 1: Distributed Architecture

2.2 Frontend Clients

To represent clients in a practical way, we built a simple frontend React web application. This frontend client allows the user to upload their images (single or multiple). The frontend client sends the request to the load balancer by making a POST request (using REST). This POST request includes the images as the payload sent to the load balancer.

2.3 Load Balancer

This architecture uses a load balancer that reroutes requests to the various master nodes we have. This load balancer has the ability to use two different algorithms to decide which master node the request needs to be rerouted to. We allow the load balancer two different algorithms to analyze the difference in performance between the two algorithms. The two load balancing algorithms will be further discussed in detail later. Since all requests are sent to the load balancer, the load balancer can serve as a bottleneck in the architecture. To increase the ability for the load balancer to handle multiple requests, the load balancer utilizes multi threading in a multi-core system.

2.3.1 Round Robin Load Balancer

The first algorithm we will analyze is a simple round robin algorithm for load balancing. Incoming requests are routed master nodes in a round robin fashion (where each master node is queued to receive the next incoming request). This algorithm does not consider the load each node experiences and does not consider the size of incoming requests. However, this algorithm is much more simple than the other algorithm we will discuss and can reroute requests at a much quicker rate. The performance differences for this trade-off will be analyzed in many scenarios. When a large request is made, there will be a difference in load between each master node and it may make more sense to evenly distribute the load across all the master nodes. However, this algorithm ignores this situation and simply routes the request to the next node in the queue. On the other hand, when there is multiple small requests that are constantly incoming, this algorithm will be much quicker at routing the requests since it does not need to consider request size and load.

Since Round Robin takes constant time (with a stored counter), we expect it to work well in cases where request sizes are similar. If request sizes are similar, then load will be evenly distributed by a round robin algorithm.

2.3.2 Lowest Load First

The next algorithm we will discuss makes use of the incoming request size (the total file size of all the images) and the total request size sent to each master node. For this algorithm to work, we maintain a vector node-loads that holds the total size of requests sent to each master node. That is, node-loads[i] denotes the total request size sent to node i . The algorithm then selects the node with the smallest total request size (That is, node-loads[0... n] is minimum). If node-loads[i] is minimum, the algorithm reroutes the request to the master node i and increments node-loads[i] with the size of the request. When finding the minimum node-loads we can decrement every entry in node-loads by the minimum to ensure that each value of node-loads does not exceed the value limit imposed by a language.

This algorithm runs in $O(n)$ time, where n is the total number of master nodes, since we have to find the minimum load node (iterate over every master node). This introduces another form of overhead for the load balancer compared to the simple round robin algorithm. However, in cases where there is a drastic difference in the size of incoming requests, this algorithm should perform better in equally balancing the nodes.

The size of the request can only be determined if the images are read by the load balancer. This introduces a challenge since this drastically increases the overhead imposed by the load balancer. The simple round robin does not read in the images, and simply reroutes the payload to a hadoop distributed file system (discussed later). This means that this algorithm is much worse in terms of performance compared to the simple round robin algorithm.

2.4 Data Flow

Images are uploaded by the client and sent to the load balancer which then reroutes the images to master nodes. The master node will then store the images in a hadoop distributed file system (HDFS) and provide workers with a URL list to access these images. This is done so images are distributed and distributed inference can be performed using Apache Spark (will be explained further later). Figure 2 below shows the data flow from the load balancer to the hadoop distributed file system. Images need to be stored on the hadoop distributed file system in order for Spark to distribute the inference across the worker nodes. The request is sent to the optimal master node depending on which load balancing technique was used (discussed previously).

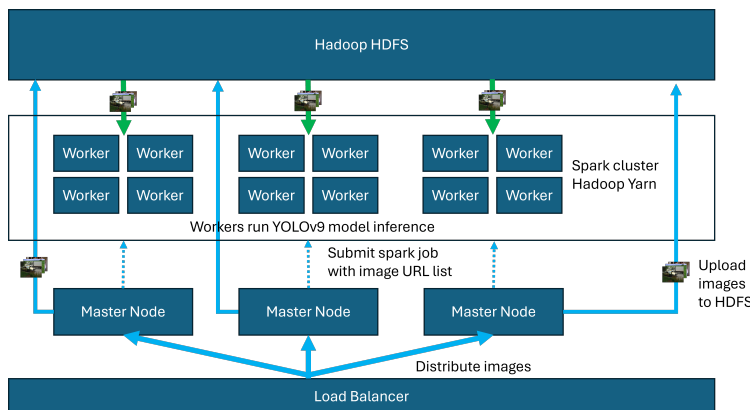


Figure 2: Data Flow

2.5 Master Nodes

Each Master node accepts incoming requests from the load balancer by utilizing a Flask server in Python. Once an incoming request comes in (with the images that need to be ran in inference), the master node uploads the images to HDFS. This ensure that the images are distributed and stored across the cluster so workers can easily access them. After storing the images, the master node generates a URL list that maps to the location of these images within the HDFS (e.g. `hdfs://hostname/image1.jpg`). Next, the master node submit a new job to the pyspark cluster and provide workers with the URL list to distribute the inference across the master and worker nodes. The pyspark cluster is managed by Apache Hadoop YARN.

The worker nodes retrieves the relevant images from HDFS using the provided URLs. Then the worker nodes run a pre-trained object detection model known as YOLOv9. This model detects objects in the images passed to it. The results are stored as JSON and show the location of an object, and the confidence of that object. The JSON file is returned to the client for it to be downloaded.

The workers runs a script that uses a pandas user-defined function (pandas UDF) within Spark to run YOLOv9 inference on images in a distributed manner. Pandas is utilized to handle the input and output of this function efficiently. It takes a pandas Series of image paths from the Spark DataFrame, processes them in batches using pandas' optimized operations, and returns a pandas Series with JSON strings of detection results, which Spark converts back into a DataFrame. This approach leverages pandas' fast data processing within Spark's distributed framework.

3 Evaluation

3.1 Setup

To emulate a distributed environment (and since inference takes lots of resources), we use a local machine to host the load balancer and the frontend client. However, the load balancer is multi threaded, so it can handle multiple concurrent requests. Using REST, a request is made from the frontend client to the load balancer (on the same machine). Then, again using REST, the load balancer redirects the request to a different machine that hosts a master node (not local) and has access to other machines (workers).

3.2 Approach

To test the performance differences between our architecture and previous solutions, we will compare the distributed architecture to a single node setup. In particular, we compare the time it takes to fulfill a request in many different scenarios. The scenarios include a single request being made, multiple concurrent requests being made (5,10), a large request being made, and multiple concurrent requests of varying sizes being made. We use these scenarios to test the difference between a single node setup and our distributed setup as well as the difference between the two load balancing algorithms previously mentioned.

To measure the time it takes to fulfill requests, we simply measure the system time on the client before sending the request and seeing the the difference to when the request returns to the client. This measurement is made using `Date.now()` in JavaScript. Because of the distributed architectures overhead with the need to pass the request from the load balancer to the master node and back, we expect for it to take longer on small requests (where the system would not be under much load). However, we expect for much better performance when the system is under load with large requests and multiple concurrent requests.

3.3 Results

Table 1 below shows the results of the tests previously discussed. The average time (in ms) to fulfill each request is shown in the table. We can see that the overhead of both Round Robin and Least Load First cause the performance to be worse using the distributed architecture compared to a single node setup. This is likely due to the large messaging overhead incurred by rerouting the request. However, as the number of files concurrently uploaded scales, the distributed architectures perform significantly better. Since all requests were of the same size, round robin distributed the load exactly the same as how Least Load First distributed the load. However, Least Load First performed worse since there is a larger overhead when computing the next node the request is sent to.

Table 1: Comparison of Computing Architectures

Requests	Architecture		
	Single Node	Distributed (Round Robin)	Distributed (Least Load First)
1	1662ms	1720ms	1922ms
5	4793ms	4280ms	6120ms
10	6978ms	4567ms	5274ms

Table 2 below shows the results of using requests of varying sizes. We do this test to show the performance benefit of Least Load First load balancing. When sizes of requests are varied, (which is likely to be the case in practice), Least Load First performs much better than the other two architectures.

Table 2: Comparison of Computing Architectures with Varying Request Size

Requests	Architecture		
	Single Node	Distributed (Round Robin)	Distributed (Least Load First)
Small	2148ms	3487ms	4473ms
Large	8463ms	6238ms	7421ms
Varying	8938ms	7794ms	6791ms

Table 3 below shows the results of using different batch sizes with a different number of images in the distributed environment. Once the quotient of the number of images and batch size becomes at least 2 (That is at least 2 batches can be sent to two different nodes), we see a significant performance increase. This is because the batches are sent to different master nodes, distributing the load.

Table 3: Distributed Architecture (RR) Performance with Different Batch and Image Sizes

Images	Batch Size						
	10	25	50	75	100	125	150
10	30,459ms	x	x	x	x	x	x
25	47,911ms	63,833ms	x	x	x	x	x
50	72,762ms	71,746ms	80,433ms	x	x	x	x
75	116,332ms	115,763ms	119,787ms	121,361ms	x	x	x
100	146,814ms	142,229ms	143,832ms	161,454ms	140,993ms	x	x
125	185,113ms	185,230ms	184,983ms	184,212ms	186,777ms	189,655ms	x
150	192,213ms	194,333ms	191,914ms	192,244ms	190,765ms	203,469ms	208,992ms

Table 4 below shows the result of the same experiment, but now in a single node environment. We see that the total request takes much longer to be fulfilled compared to the distributed environment. We also see that the performance gets worse when the quotient of the number of images and batch size becomes greater than or equal to 2. This is due to the communication overhead and the fact that the work does not become distributed.

Table 4: Single Node Performance with Different Batch and Image Sizes

Images	Batch Size						
	10	25	50	75	100	125	150
10	34,132ms	x	x	x	x	x	x
25	87,691ms	83,562ms	x	x	x	x	x
50	169,432ms	167,624ms	165,888ms	x	x	x	x
75	238,998ms	237,542ms	237,184ms	239,552ms	x	x	x
100	306,757ms	303,898ms	313,479ms	314,633ms	312,554ms	x	x
125	394,452ms	395,763ms	391,996ms	392,654ms	389,773ms	362,653ms	x
150	459,224ms	467,334ms	468,939ms	449,382ms	455,722ms	447,899ms	439,842ms

4 Conclusion

In this paper, we discussed a distributed architecture to efficiently perform object detection from client requests. This architecture is scalable and able to handle multiple concurrent requests. The architecture uses a load balancer that has access to two different load balancing techniques(Round Robin, Least Load First).This load balancer then selects the optimal master node to send the request to. The master nodes perform inference using a pretrained model YOLOv9 and distributes the inference across multiple worker nodes by using Apache Spark.

In different scenarios, the distributed architecture had different performance compared to a single node system. However, at scale, the distributed system performed much better. There were situations where Round Robin load balancing performed better than Least Load First load balancing and there were other solutions where the latter performed better. Overall, we analyzed different situations that were better suited for each algorithm.

5 References

- [1] Apache Spark 3.5.1: <https://spark.apache.org/docs/3.5.1/>
- [2] YOLOv9: <https://github.com/WongKinYiu/YOLOv9>
- [3] P.P. Geethu Gopinath, Shriram K. Vasudevan, An In-depth Analysis and Study of Load Balancing Techniques in the Cloud Computing Environment, Procedia Computer Science, Volume 50, 2015, <https://doi.org/10.1016/j.procs.2015.04.009>. (<https://www.sciencedirect.com/science/article/pii/S1877050915005104>)